



Fakultät Informatik

From CAD Software to Slicer

Workflow Optimization using Print Hints in 3MF Files for 3D Printing

Bachelorarbeit im Studiengang Medieninformatik

vorgelegt von

Luise Hartdegen

Matrikelnummer 3642168

Erstgutachter: Prof. Dr. Ulrich von Zadow

Zweitgutachter: Prof. Dr. Timo Götzelmann

© 2025

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Abstract

Kurzdarstellung

//TODO

Contents

1. Introduction	1
2. Problem	3
3. Background	4
3.1. State of the Art Workflow	4
3.1.1. Exporting as STL	4
3.1.2. Exporting as OBJ	5
3.1.3. Exporting as 3MF	5
3.1.4. Applying Settings with and without Presets	5
3.2. Foundations	6
4. Method	9
4.1. Explored Possible Solutions	9
4.2. Fusion Add-In	9
4.2.1. Script vs. Add-in	9
4.2.2. Python vs. C++	10
4.3. if necessary: changes to 3MF File Format	10
4.4. slicer changes	10
4.5. Implementation Design	10
5. Results	11
6. Conclusion	12
6.1. Outlook	12
6.2. Summary	12
A. Supplemental Information	13
List of Figures	15
List of Tables	16
List of Listings	17
Bibliography	18

Chapter 1.

Introduction

It is one of the clearest paradoxes of modern work that computer-based systems designed to reduce task complexity and cognitive workload actually often impose even greater demands and stresses on the very individuals they are supposed to be helping.[1, p. 222]

When designing a model for 3D printing, the process frequently follows a certain set of iterative steps. After the ideation phase, the design engineer first creates the model using CAD software, imports the file (e.g. STL, 3MF) to a slicer software, sets the necessary parameters and finally starts printing. They evaluate the result for design flaws or areas of improvement, leading to model revisions and thereby reinitiating the workflow as pictured in figure 2.1. Throughout the iterations, some printing parameters will be tweaked but overall you want to keep them consistent to have a reliable comparison. Despite automation and optimization having been a prevalent topic in the software developer community for years, setting the parameters is generally a manual step. Hence, the design engineer has to remember their settings or save them some other way, possibly by writing them down meticulously or taking screenshots.

With access to printers in public institutions becoming easier and the growing popularity of 3D printing not only within the *maker* community but in casual users as well, HCI needs to focus on streamlining error-prone workflows like the one described above[2]. “[C]urrent tools that focus on supporting only one aspect of the workflow (e.g. , only modelling or only printing) may be less useful for casual makers and there is opportunity in HCI research to innovate on design tools that take into account the interdependencies within the 3D printing workflow”[2]

This thesis focuses on the communication between CAD software and slicer, with a particular focus on proving the feasibility of ‘print hints’. The goal is to streamline the iterative design workflow by reducing the need to repeatedly configure the same printing parameters in each iteration, therefore lowering cognitive workload for design engineers. The approach as currently planned contains two core components: firstly, an improved export function within the CAD software *Autodesk Fusion*, e.g. to mark certain objects

as modifiers; and second, automatic handling of materials and these print hints within the open source slicer software called *OrcaSlicer*.

basic (HCI) aspects in which I want to improve the workflow (possibly to be referenced in the following texts) → what does it solve

- sharing files with slicer but not printer specific settings
- how it reduces cognitive workload in repetitive steps

“*makers*, a community of enthusiasts who focus on fabrication, invention and experience sharing, and who collaborate and learn in environments known as *makerspaces*”[2]

“thousands of public institutions, such as schools, libraries, and universities are now setting up print centers or creative hubs for end users to make use of digital fabrication technologies”[2]

“makerspaces that are often run by enthusiasts [...] attract engineers, entrepreneurs, inventors, hackers, craftsmen, and artists”[2]

“casual makers serve as a proxy for what it will be like for the general public to use 3D printing”[2]

“if HCI is going to be at the forefront of inventing new fabrication design tools and interfaces[...], we need to look beyond enthusiasts and understand the challenges and opportunities that exist in facilitating the interactions of casual makers with fabrication technology”[2]

“current tools that focus on supporting only one aspect of the workflow (*e.g.* , only modelling or only printing) may be less useful for casual makers and there is opportunity in HCI research to innovate on design tools that take into account the interdependencies within the 3D printing workflow”[2]

“while lowering the usability barriers of fabrication tools is important, it is perhaps even more important to weave-in expertise on best practices and troubleshooting tips within the walk-up-and-use tools used by casual makers to reduce their dependency on operators”[2]

“opportunities that exist in building the next generation of inter-connected fabrication tools with features for supporting expertise sharing”[2]

“enthusiasts and professional users—that is, they were trained or technically literate in fabrication”[2]

Chapter 2.

Problem

what's the real world problem I am trying to solve? why is solving it important?

- Repetitive Steps in the Iterative Workflow

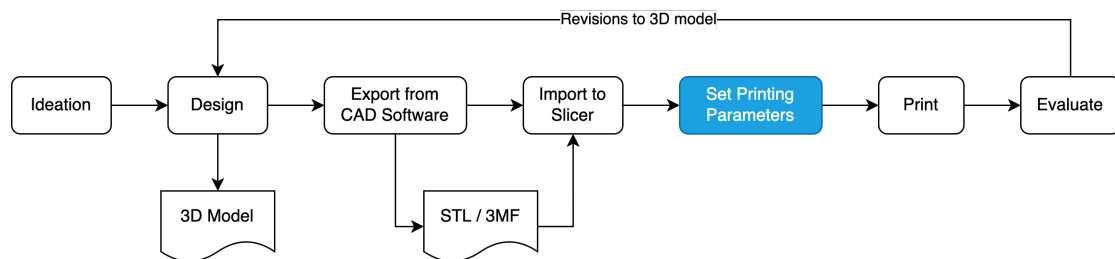


Figure 2.1.: Iterative workflow in 3D printing highlighting the step to be optimized

- Negative Effects resulting from manual Repetitive Steps

//TODO research cognitive workload / increased amount of errors due to necessary repetitive steps (podcast about to-do lists → air plane take off / landing checklists (yes, this is relevant))

some HCI foundational work if necessary

“If the user’s print failed or did not match their design intent, they would have to adjust either their model or print settings to correct the issue, often with the operator’s assistance”[2]

- Usage of Presets

do people (esp. beginners) use presets? if no, why and how would it make their life easier? is it even possible in community spaces like libraries and schools?

- the need for elaborate and extensive printing instructions when sharing a complex model online

Chapter 3.

Background

3.1. State of the Art Workflow

When analyzing the process from idea to print, several main steps can be identified as shown in 2.1. Using a simple box with a snap-fit closure and hinge as an example, the following analysis will focus on the steps from exporting a model to printing, highlighting in particular how the workflow differs depending on the file format used in regards to three points.

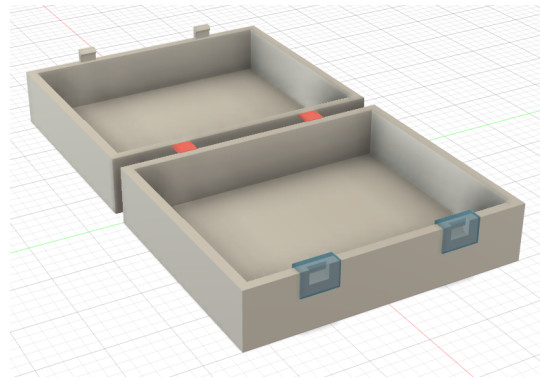


Figure 3.1.: 3D model of simple box

1. A model consisting of multiple parts
2. Using multiple materials (e.g. printing the hinge in a flexible material)
3. Using model parts as modifiers (e.g. the light blue boxes as modifiers to print more wall loops in this area)

Exporting the model from Fusion is possible via two dialogs: the general export and the 3D print specific one. The two dialogs each offer a different set of file formats due to their different purposes. Since this analysis deals with 3D printing in particular, only the export dialog specific to 3D printing will be considered, which offers to export as STL, 3MF and OBJ format. For all three the user can decide whether to generate one file containing all selected components and bodies, or one file per body.

3.1.1. Exporting as STL

Exporting the model as an STL file immediately shows one important distinction. Namely, that there will always be one body in the resulting file, even if the model is comprised of multiple bodies within Fusion. Therefore the user has to choose whether

to combine all bodies into one, and thus losing the option to print the hinges in a flexible material or to use parts as modifiers (e.g. the blue boxes for additional wall loops). Alternatively exporting each body as a separate file might lose their spatial relations to each other while keeping the bodies separate. However, “STL native files describe only the surface geometry of a three dimensional object without any representation of color, texture or other common CAD model attributes.”[3] Even if a user applies materials in Fusion, that information won’t be exported, and the user has to apply all materials anew in OrcaSlicer.

3.1.2. Exporting as OBJ

The OBJ file format “is the preferred 3D printer file format for multicolor printing”.[3] Consequently OrcaSlicer shows a particular dialog (fig. A) when opening an OBJ file, asking the user how they want to map the file’s colors to current or new filament presets. This makes it easy for users to align filament colors (i.e. the color the filament is represented as in OrcaSlicer, not necessarily the color of the filament itself) to colors applied in Fusion.

Just like STL files though, the model is generated as one body per file resulting in the same disadvantages regarding the use of model parts as modifiers. Additionally, if a user accidentally deletes one of the filament colors, there’s no way to select the respective parts to reapply presets. Instead the user would have to reimport the entire model, thus potentially losing other settings they have already applied.

3.1.3. Exporting as 3MF

Unlike the other two formats, 3MF preserves the separate bodies even when exported as a single file, making it useful for applying different materials, settings or modifiers to each body, while safely keeping the spatial relations.

//TODO TBC

3.1.4. Applying Settings with and without Presets

Generally, materials can be set using system or user defined presets per object. Material settings entail for example the filament itself, nozzle temperature, general printing speed and cooling settings. OrcaSlicer actively asks the user, whether they want to save any changes to material parameters they have made, which makes it easy to save either parameters specific to one’s printer and filament or even specific to one project while simultaneously keeping default material presets untouched.

Furthermore settings like the amount of wall loops, infill percentage and pattern, and printing speed for single layers might need to be customized. It is possible to save these as presets as well, however only globally, i.e. they can only be applied to the entire project, not to specific objects within. As criticized by GitHub user *headamame* in the [4, OrcaSlicer Issue #7769], particularly for projects containing multiple objects, using presets is not a valid option.

3.2. Foundations

- what is CAD software?
- what is a slicer?

“Processing the 3D model occurred in specialized software known as a slicer (as it slices the model into layers for the printer), which allowed users to select print settings, such as layer height and print speed”[2].

- 3MF File Format → why 3MF and not STL or other file formats? why not OBJ, since it supports exporting materials from fusion and mapping then in OrcaSlicer natively?

3.2.0.1. The 3MF specification

[5]

- “3MF Document markup has been designed in anticipation of the evolution of this specification. It also allows third parties to extend the markup. Extensions are a critical part of 3MF, and as such, this core specification is as narrow as possible.” [p. 8]
- 3MF = ZIP archive following the .ZIP File Format Specifications by PKWARE Inc. containing one or more 3D payloads
- 3D payload = “complete collection of interdependent parts and relationships within a package” consisting of “a 3D object definition and its supporting files”
- 3MF document parts → table 3.1, figure 3.2
- PrintTicket → seems to be OS specific, so probably not helpful but *//TODO* investigate further

Name	Description	Required / Optional
3D Model	Contains the description of one or more 3D objects for manufacturing.	Required
Core Properties	The OPC part that contains various document properties.	Optional
Object Thumbnail	Contains a small JPEG or PNG image that represents a 3D object in a 3D Model.	Optional
3D Texture	Contains a texture used to apply color to a 3D object in the 3D Model part (available for extensions)	Optional
PrintTicket	Provides settings to be used when outputting the 3D object(s) in the 3D Model part.	Optional
Custom Parts	OPC parts that are associated with meta-data	Optional

Table 3.1.: Excerpt of 3MF document parts[5, p. 4]

```

example.3mf
├── _rels/
│   ├── [Content_Types].xml/
│   ├── 3D/
│   │   └── 3dmodel.model
│   └── Metadata/
│       └── thumbnail.png

```

Figure 3.2.: Typical internal structure of 3MF file as exported from *Autodesk Fusion*

- most important: 3D Model Part = root of 3D payload → “contains definitions of one or more objects to be fabricated by 3D manufacturing processes”[5, p. 6] → contains resource definitions (objects, materials) and items to be build

- there MUST be exactly one <model> element in a 3D Model part
- a model must have two additional child elements: <resources> and <build>
- <resources> element

provides a set of definitions that can be drawn from to define a 3D object; root element of a library of constituent pieces of the overall 3D object definition → objects, properties, materials; ‘object resource’: 3d object that could be but will not be manufactured on its own

- <build> element

provides a set of items that should actually be manufactured as part of the job; contains `<item>` elements = objects to be manufactured

Chapter 4.

Method

what's the solution(s) I came up with → sketching out different possibilities and why I chose the one I did

4.1. Explored Possible Solutions

4.2. Fusion Add-In

- what is CAD software
- a lil about their API: add-in vs. script, Python vs. C++
- why I chose to realise it as a Python add-in
- possibly different approaches if more than one way to solve it came up

As mentioned before, the software supports developers to create additional functionalities by providing an extensive API and the option to create either an add-in or a script directly from within Fusion's UI. Using this, Fusion sets up the project structure as well as some example code to make it easier to get started especially for beginners. Fusion runs natively on Python but offers to create the add-in or script either in Python or in C++.

4.2.1. Script vs. Add-in

[6]

- **Run on Startup** – This setting is add-in-specific and indicates if the add-in should be run automatically when Fusion is started. Most add-ins will want to take advantage of this capability so the commands they define will be available to the user as soon as Fusion starts.

- The user executes a script through the “Scripts and Add-Ins” command, and the execution stops immediately after the run function is complete. A script runs, and then it is done.
- An add-in is typically automatically loaded by Fusion when Fusion starts up. An add-in also usually creates one or more custom commands and adds them to the user interface during startup. The add-in continues to run throughout the Fusion session, so it can react whenever the user executes any of its commands. The add-in remains running until Fusion is shut down or the user explicitly stops it through the “Scripts and Add-Ins” dialog. When it stops, it cleans up whatever user interface customization it created in its stop function.

4.2.2. Python vs. C++

4.3. if necessary: changes to 3MF File Format

4.4. slicer changes

- how does it process 3mf per default?
- different possible approaches to solve the problem at hand and why did I do it the chosen way?

4.5. Implementation Design

- software architecture (diagram?)
- input, output
- choices that had to be made

Chapter 5.

Results

- what does the optimised workflow look like?
- why is the solution good?
 - how are the results analysed / validated?
 - openly available → advantages of that
 - but: it is a proof of concept / prototype and thus will most likely not be the end of the story

Chapter 6.

Conclusion

6.1. Outlook

- communication with OrcaSlicer's owner softfever?
- option of code being pulled into other slicers
- what is left to do?

6.2. Summary

Appendix A.

Supplemental Information

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <model xmlns="http://schemas.microsoft.com/3dmanufacturing/core
  /2015/02" unit="millimeter" xml:lang="en-US" xmlns:m="http:
  //schemas.microsoft.com/3dmanufacturing/material/2015/02"
  xmlns:p="http://schemas.microsoft.com/3dmanufacturing/
  production/2015/06" xmlns:b="http://schemas.microsoft.com/3
  dmanufacturing/beam lattice/2017/02" xmlns:s="http://schemas.
  microsoft.com/3dmanufacturing/slice/2015/07" xmlns:sc="http:
  //schemas.microsoft.com/3dmanufacturing/securecontent
  /2019/04">
3 <metadata name="Title" type="QName" preserve="1">Test v2</
  metadata>
4 <resources>
5   <m:colorgroup id="2">
6     <m:color color="#CC60FFFF" />
7     <m:color color="#A0A0A0FF" />
8   </m:colorgroup>
9   <object id="1" name="Body9" type="model" p:UUID="33f607be
    -287f-48ea-b893-8ee1626ed2b1" pid="2" pindex="0">
10     <mesh>
11       <vertices>
12         <vertex x="-39.484996" y="6.249999" z="30.000000" />
13         <vertex x="-37.568216" y="6.249999" z="30.000000" />
14         <vertex x="-37.568216" y="6.249999" z="33.000000" />
15         ...
16       </vertices>
17       <triangles>
18         <triangle v1="0" v2="1" v3="3" />
19         <triangle v1="3" v2="1" v3="2" />
20         <triangle v1="4" v2="0" v3="5" />
```

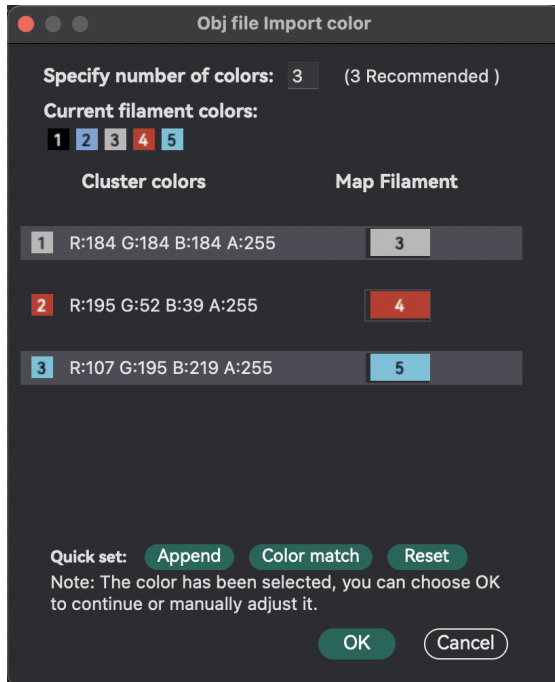


Figure A.1.: Dialog when opening an OBJ file in OrcaSlicer to map colors to filament presets

```

21      ...
22      </triangles>
23    </mesh>
24  </object>
25 </resources>
26 <build p:UUID="cd9553e5-fa90-4b4e-916e-bd1fed06979c">
27   <item objectid="1" p:UUID="91670927-ae59-46f3-cb97-91
      a310462033"/>
28 </build>
29 </model>

```

Listing A.1: Abbreviated Example of 3MF's 3dmodel.model File

List of Figures

2.1. Iterative workflow in 3D printing highlighting the step to be optimized	3
3.1. 3D model of simple box	4
3.2. Typical internal structure of 3MF file as exported from <i>Autodesk Fusion</i> . . .	7
A.1. Dialog when opening an OBJ file in OrcaSlicer to map colors to filament presets	14

List of Tables

3.1. Excerpt of 3MF document parts	[5, p. 4]		7
------------------------------------	-----------	-----------	---

List of Listings

A.1. Abbreviated Example of 3MF's 3dmodel.model File	13
--	----

Bibliography

- [1] J. A. Jacko, *Human Computer Interaction Handbook: Fundamentals, Evolving Technologies, and Emerging Applications, Third Edition*. Milton, UNITED KINGDOM: Taylor & Francis Group, 2012. [Online]. Available: <http://ebookcentral.proquest.com/lib/thnuernberg/detail.action?docID=911990>
- [2] N. Hudson, C. Alcock, and P. K. Chilana, “Understanding Newcomers to 3D Printing: Motivations, Workflows, and Barriers of Casual Makers,” in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 384–396. [Online]. Available: <https://dl.acm.org/doi/10.1145/2858036.2858266>
- [3] C. Iancu, “ABOUT 3D PRINTING FILE FORMATS,” *Annals of the Constantin Brancusi University of Targu Jiu*, no. 2, pp. 135–138, 2018.
- [4] headamame, “Copy/paste per-object settings · Issue #7769 · SoftFever/OrcaSlicer,” Dec. 2024. [Online]. Available: <https://github.com/SoftFever/OrcaSlicer/issues/7769>
- [5] M. Consortium, “3MF Core Specification,” Feb. 2025. [Online]. Available: <https://3mf.io/spec/core-v1-3-0/>
- [6] S. F. Autodesk Inc., CA, “Fusion Help | Creating a Script or Add-In | Autodesk,” 2025. [Online]. Available: <https://help.autodesk.com/view/fusion360/ENU/?guid=GUID-9701BBA7-EC0E-4016-A9C8-964AA4838954>
- [7] —, “Fusion API Reference Manual,” 2025. [Online]. Available: <https://help.autodesk.com/view/fusion360/ENU/?guid=GUID-7B5A90C8-E94C-48DA-B16B-430729B734DC>
- [8] A. W. Kushniruk, M. M. Triola, E. M. Borycki, B. Stein, and J. L. Kannry, “Technology induced error and usability: the relationship between usability problems and prescription errors when using a handheld application,” *International Journal of Medical Informatics*, vol. 74, no. 7-8, pp. 519–526, Aug. 2005.
- [9] SoftFever, RF47, ianalexis, and Arcodude, “OrcaSlicer/doc.” [Online]. Available: <https://github.com/SoftFever/OrcaSlicer/tree/main/doc>

- [10] B. Subbaraman, N. Bursch, and N. Peek, “It’s Not the Shape, It’s the Settings: Tools for Exploring, Documenting, and Sharing Physical Fabrication Parameters in 3D Printing,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’25. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 1–19. [Online]. Available: <https://dl.acm.org/doi/10.1145/3706598.3713354>
- [11] J. Walter-Herrmann, *FabLab: Of Machines, Makers and Inventors*, 1st ed., ser. Kultur- und Medientheorie. Bielefeld: transcript, 2014.
- [12] A. Wiberg, J. Persson, and J. Ölvander, “Design for additive manufacturing – a review of available design methods and software,” *Rapid Prototyping Journal*, vol. 25, no. 6, pp. 1080–1094, Jul. 2019. [Online]. Available: <https://www.emerald.com/insight/content/doi/10.1108/RPJ-10-2018-0262/full/html>
- [13] Z. Yan, M. S. Dhaygude, and H. Peng, “Make Making Sustainable: Exploring Sustainability Practices, Challenges, and Opportunities in Making Activities,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’25. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 1–14. [Online]. Available: <https://dl.acm.org/doi/10.1145/3706598.3713665>